

# Documentación del Pipeline de Datos

## Examen Técnico — Data Engineer Semi Senior

Fecha: 2026-06-17

### 1. Resumen Ejecutivo

Este documento describe el pipeline de datos diseñado para el caso técnico de Financiera OH. El pipeline cubre tres capas: análisis SQL sobre datos de clientes y transacciones, integración y estandarización de tarjetas de dos procesadores externos, y validación de calidad sobre un dataset de transacciones anómalas.

### 2. Arquitectura General del Pipeline

El pipeline se divide en 3 partes independientes:

| Parte   | Descripción                                        | Entrada                                                          | Salida                                                       |
|---------|----------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------|
| Parte 1 | Análisis SQL de clientes, tarjetas y transacciones | customers, cards, transactions                                   | Reportes analíticos                                          |
| Parte 2 | Integración de procesadores y cálculo financiero   | cards_processor_a/b, equivalence_table_extended, financial_rules | cards_financial_master                                       |
| Parte 3 | Data Quality Report y limpieza de anomalías        | transactions_anomalies                                           | data_quality_report.json<br>transactions_anomalies_clean.csv |

### 3. Fuentes de Datos (Tablas de Entrada)

| Tabla                      | Columnas Clave                                                     | Descripción                                                           |
|----------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| customers                  | customer_id, full_name, segment, register_date                     | Maestro de clientes con segmentación Bronze / Silver / Gold           |
| cards                      | card_id, customer_id, card_type, status                            | Tarjetas asociadas a clientes (Mastercard / Visa)                     |
| transactions               | tx_id, card_id, amount, tx_date                                    | Transacciones financieras por tarjeta                                 |
| cards_processor_a          | card_id_a, customer_id, card_type_a, status_a                      | Tarjetas del procesador externo A (valores crudos: GOLD_A, ACTIVE_A)  |
| cards_processor_b          | card_id_b, customer_id, card_type_b, status_b                      | Tarjetas del procesador externo B (valores crudos: GOLD_B, ENABLED_B) |
| equivalence_table_extended | attribute, value_processor_a, value_processor_b, value_standard    | Tabla de homologación de valores entre procesadores                   |
| financial_rules            | processor, card_type_raw, interest_rate, commission, insurance_fee | Reglas financieras por procesador y tipo de tarjeta                   |
| transactions_anomalies     | tx_id, card_id, amount, tx_date, extra_column                      | Dataset con anomalías intencionales para validación de calidad        |
| business_rules_text        | rule_id, description                                               | Reglas de negocio en lenguaje natural para traducción a SQL           |

### 4. Parte 1 — Análisis SQL

Se desarrollaron 6 consultas SQL sobre las tablas customers, cards y transactions. Las queries utilizan SQL estándar compatible con BigQuery.

| # | Consulta | Técnica SQL | Resultado |
|---|----------|-------------|-----------|
|---|----------|-------------|-----------|



## 6. Parte 3 — Data Quality Report

Script Python con pandas que valida transactions\_anomalies.csv, genera un reporte JSON con los problemas detectados y exporta el dataset limpio.

### 6.1 Validaciones aplicadas

| Validación                           | Método                       | Resultado                                             |
|--------------------------------------|------------------------------|-------------------------------------------------------|
| Valores nulos por columna            | isnull().sum()               | tx_date: 9   amount: 5   card_id: 4   extra_column: 9 |
| Filas duplicadas                     | duplicated().sum()           | 0 duplicados                                          |
| Fechas futuras (>hoy)                | pd.to_datetime + comparación | 5 transacciones con fecha futura                      |
| card_id inexistentes en procesadores | set.union() + isin()         | 7 filas inválidas (X9999 + nulls)                     |
| Campos no documentados               | set difference de columnas   | extra_column no está en el esquema                    |
| Montos negativos                     | amount < 0                   | 9 transacciones con monto negativo                    |

### 6.2 Proceso de limpieza

- Eliminar filas con nulls en card\_id, amount o tx\_date.
- Eliminar filas con fechas futuras (tx\_date > hoy).
- Eliminar filas con card\_id inexistentes en los procesadores.
- Eliminar filas con amount negativo.
- Eliminar columna extra\_column (no documentada).
- Resultado: 1 fila limpia de 20 originales — el dataset era prácticamente todo anomalías.

### 6.3 Archivos generados

| Archivo                          | Descripción                                                        |
|----------------------------------|--------------------------------------------------------------------|
| data_quality_report.json         | Reporte JSON con todos los problemas detectados por categoría      |
| transactions_anomalies_clean.csv | Dataset limpio listo para carga a BigQuery                         |
| load_bq.py                       | Script opcional para carga a BigQuery usando google-cloud-bigquery |

## 7. Conclusiones y Recomendaciones

### Conclusiones

- El pipeline cubre correctamente las 3 capas: análisis, integración y calidad de datos.
- Los datos de la Parte 1 están íntegros: no hay transacciones huérfanas ni inconsistencias graves.
- La integración de procesadores requiere tabla de equivalencias para evitar valores crudos en capas de BI.
- El dataset de anomalías tiene una tasa de error del 95% — indica que es un archivo de prueba, no productivo.

### Recomendaciones

- Implementar la tabla cards\_financial\_master en BigQuery con PARTITION BY load\_date y CLUSTER BY processor, customer\_id para optimizar costos.
- Formalizar las reglas de negocio (business\_rules\_text) en tablas estructuradas antes de implementarlas en SQL.
- Agregar columna issue\_date a cards\_processor\_b para habilitar la Regla 2 (LEGACY\_SILVER).

- Definir el valor numérico de "comisión reducida" y "beneficio premium" en financial\_rules.
- Automatizar el Data Quality Report como paso previo a cualquier carga a BigQuery.